

Real-Time Multiplayer Quiz Game

using TCP & UDP

CS411 – NETWORK
SOCKET PROGRAMMING
AND TRANSPORT
PROTOCOL DESIGN

PRESENTED BY
AHMED HAMOUDA
YASSINE MTIBAA
YOUSSEF BEN
MOUSSA
AYMEN SAAD

Table of Contents

INTRODUCTION

**SYSTEM
ARCHITECTURE
OVERVIEW**

**IMPLEMENTATION
HIGHLIGHTS**

**PERFORMANCE
COMPARISON:
TCP VS UDP**

**CHALLENGES &
SOLUTIONS**

**CONCLUSION &
FUTURE WORK**

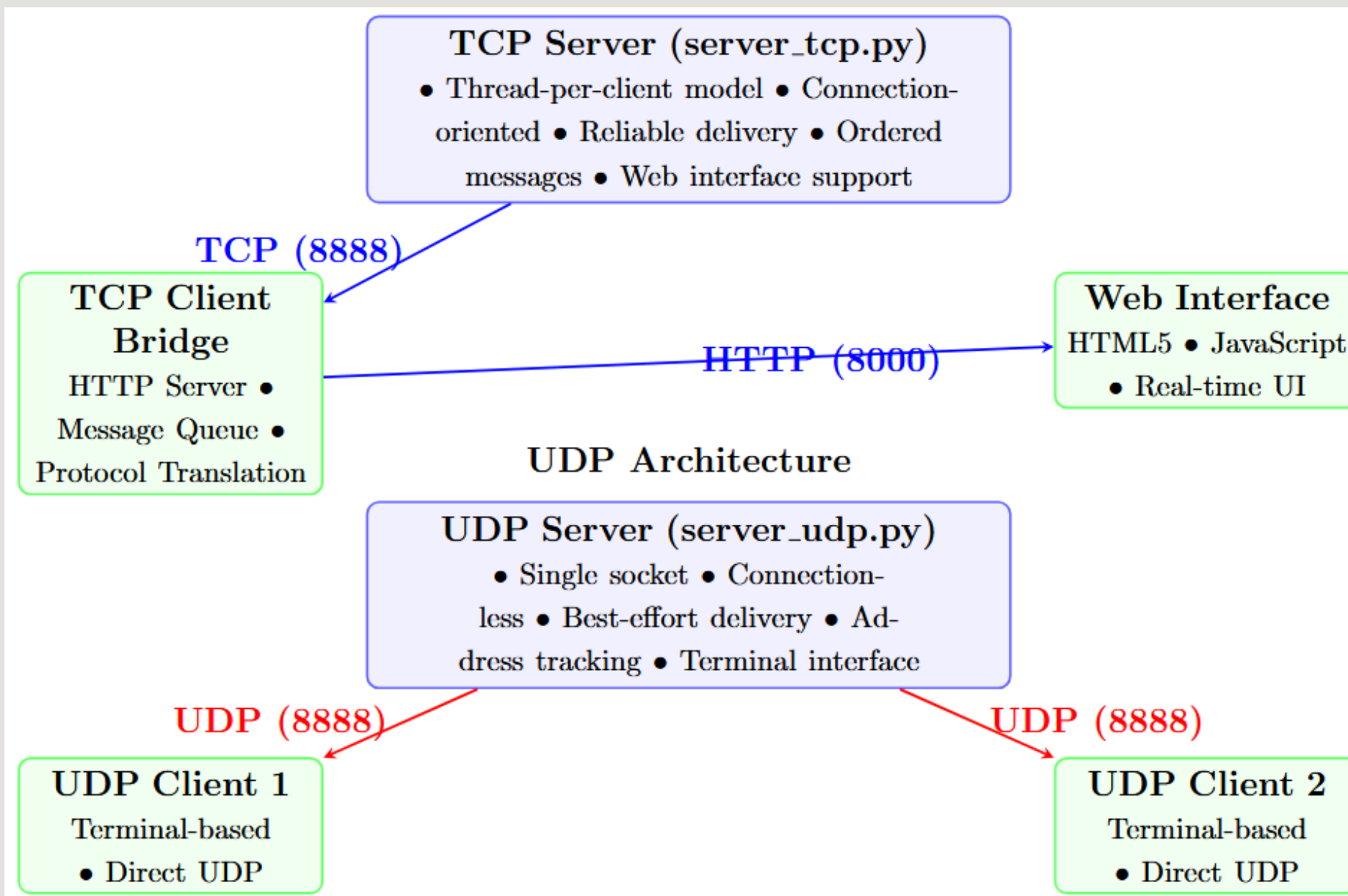
Introduction

01

- Designed and compared TCP and UDP versions of a real-time quiz game using Python.
- Explored connection-oriented vs. connectionless communication.

System Architecture Overview

02



Two architectures with identical game logic:

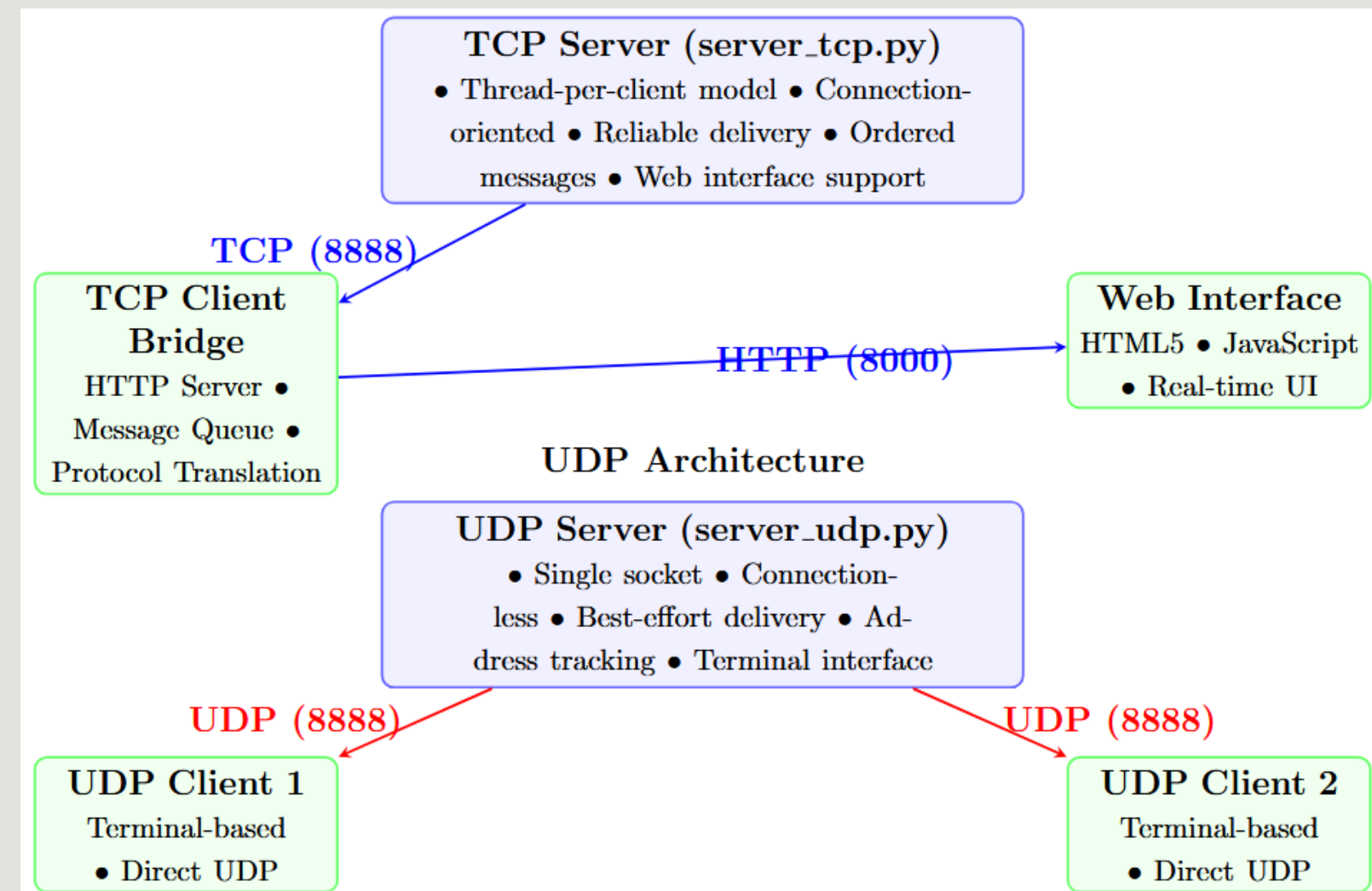
- **TCP:** connection-oriented, reliable, multi-threaded, web interface.
- **UDP:** connectionless, lightweight, single-threaded, terminal-based.

Enabled direct comparison between protocol behaviors.

Implementation Highlights

03

- **Unified game flow:** join, question, scoring, leaderboard.
- **TCP ensures** ordered, reliable message delivery (stream-based).
- **UDP uses** datagrams with manual acknowledgments and retransmissions.
- Same time-based scoring algorithm ensures fairness across protocols.



Performance Comparison: TCP vs UDP

04

- UDP: faster response times, lower overhead, more scalable.
- TCP: perfect reliability, automatic session management.
- TCP is ideal for quiz or financial apps.
- UDP is better for games, streaming, or voice applications.

Challenges & Solutions

- TCP: Managed concurrent connections safely with thread locks.
- UDP: Implemented application-level acknowledgment and retransmission.
- Abstracted game logic to maintain consistency across protocols.

Conclusion & Future Work

05

- Achieved dual protocol design and detailed performance analysis.
- TCP prioritized reliability; UDP prioritized speed and scalability.
- Enhanced understanding of real-world transport protocol trade-offs.

Future goals:

- Implement QUIC for hybrid performance.
- Add TLS/DTLS encryption.
- Develop mobile-optimized clients.

THANK YOU!